

Public Twitter/X Feed Viewer Prototype

Chrome extension, FastAPI session services, MongoDB, feed retrieval, and a security-first redesign

Project context: Google Developer Student Club, IIT Roorkee project with a working demo and public repository

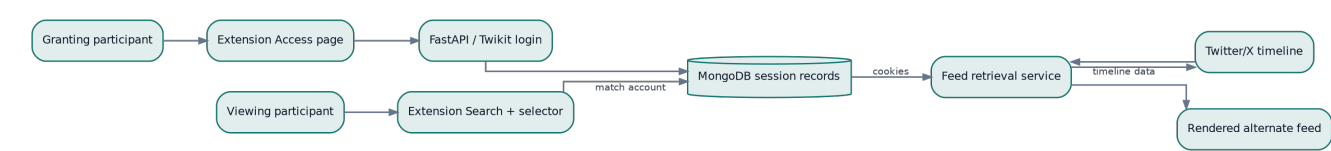
Status: Implemented academic prototype

A browser-extension prototype that lets one participant grant access to a representation of their Twitter/X home feed and lets another participant switch between available feed views. The implemented system combines extension pages, a FastAPI/Twikit backend, MongoDB session records, cookie-based feed retrieval, and explicit session-expiry handling. Because the prototype accepts credentials and stores session cookies, this work sample also documents the safer production architecture that should replace that mechanism.

Technology stack

Chrome Extension	JavaScript / React build	Python	FastAPI
Twikit	MongoDB	Cookies / sessions	Browser storage

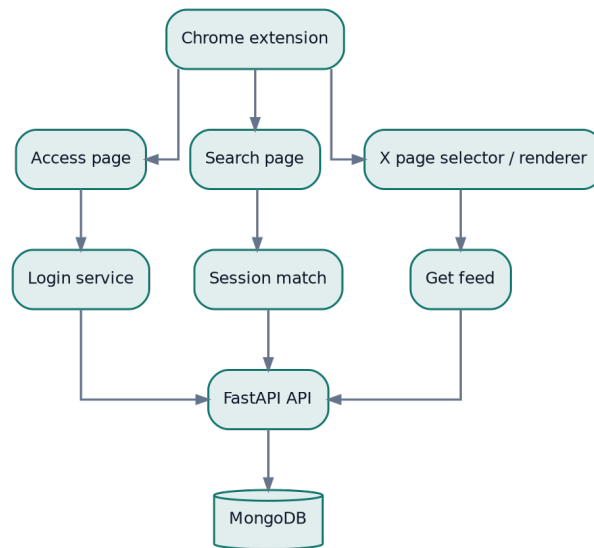
System at a glance



Access and discovery UI Extension pages collect an access grant and let a viewer search an available account by username or email.	Session matching The backend associates an account reference with stored session material and returns a selected session to the extension.
Feed retrieval A feed service uses the selected session to request timeline items including text, images, video, likes, and comments.	Expiry handling Expired session material is removed and the interface asks the granting participant to establish a new session.

System design and component ownership

The project has two clear runtime halves. The extension owns user interaction and feed switching inside the browser; the Python service owns login/session operations and timeline retrieval. MongoDB connects the grant step to the later viewing step. The architecture is technically interesting, but the implemented credential and cookie boundary is appropriate only for a controlled prototype.



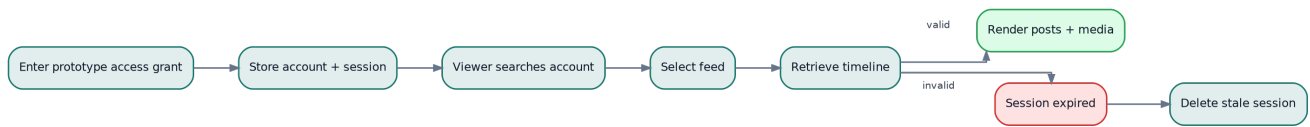
Component	Responsibility
Extension popup	Provides Access and Search entry points.
Access page	Collects the account reference and initiates the prototype login flow.
Search and selector UI	Finds available account references and stores the selected session in extension storage.
X page integration	Shows a selector on the home feed and renders the chosen timeline data.
FastAPI / Twikit services	Implement login, session matching, expiry handling, and feed retrieval.
MongoDB session records	Associate account references with authentication cookies for later lookup.

Key design decisions

- Separate browser integration from Python session/feed logic.
- Store only a selected session reference in extension storage after lookup.
- Handle session expiry explicitly instead of silently returning an empty feed.
- Preserve media and engagement metadata so the alternate feed resembles the source timeline.

Core execution flow

The implemented prototype starts with an access grant, persists session material, and later resolves it for a viewer. The viewer selects an available account from the extension, after which the backend retrieves and normalizes that timeline. Failure is treated as an explicit session-expired state.



Implementation notes

- The README documents separate login, match-session, and get-feed backend responsibilities.
- The extension contains Access and Search pages plus an X-home-page selector.
- The selected cookies are placed in extension storage for the feed request.
- Feed results include text, images, videos, likes, and comment counts.
- When a session is expired, the backend removes the stored cookies and returns a re-authentication message.

Representative service contract

```
@app.post("/sessions/grant")
async def grant_access(request: GrantRequest):
    session = await provider_login(request.username, request.password)
    await sessions.save(request.account_ref, session.cookies)
    return {"status": "available"}

@app.get("/feeds/{account_ref}")
async def read_feed(account_ref: str):
    cookies = await sessions.load(account_ref)
    try:
        return await timeline.fetch(cookies)
    except SessionExpired:
        await sessions.delete(account_ref)
        raise HTTPException(401, "session_expired")
```

This compact contract represents the documented login, session-match, feed, and expiry behavior. It is not a verbatim source file and must not be interpreted as a recommended production authentication design.

Engineering review and production path

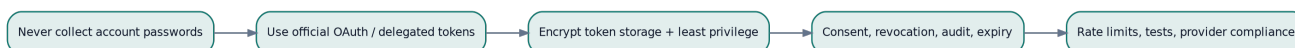
Engineering strengths

- Combines a browser extension with a Python service and persistent session lookup.
- Handles structured timeline data across text, image, video, and engagement fields.
- Makes session expiry a recoverable product state.
- Provides both a public repository and a recorded working demonstration.

Current limitations

- The prototype asks a participant to enter an X password into the extension flow.
- Authentication cookies are stored in MongoDB and browser extension storage.
- The implementation depends on an unofficial integration and may conflict with platform terms or interface changes.
- The documented flow lacks encryption-at-rest details, granular revocation, audit logs, rate limits, and automated tests.

Recommended next steps



Why this is relevant to AI training work

- Creates difficult evaluation cases around permission, session expiry, data normalization, and partial media records.
- Shows how to inspect an implemented system and distinguish functionality from unsafe design choices.
- Supports tasks for rewriting an insecure authentication boundary into a safer delegated-access design.
- Requires clear reasoning about browser state, backend state, and provider failures.

Repository:

https://github.com/SidheshwarSarangal/X_feed_extension

Scope and provenance:

Prepared from the public repository README, project summary in the supplied resume, and repository history. The security review is intentionally candid: the current password/cookie method is described as prototype behavior, while the roadmap recommends official delegated access.

